

Competitive Programming

Section 57: Reductions And Modeling

Competitive Programming Course

2026-07-10

Table of contents

1	Turning strange into known	3
1.1	Reduction workflow	3
1.2	Bipartite matching model	3
1.3	2-SAT model	3
1.4	Flow model clues	4
1.5	Modeling checklist	4
1.6	2-SAT mini-example	4
1.7	Matching reduction example	4
1.8	Flow modeling mini-pattern	5
1.9	Reduction proof template	5
2	Exercises	6
2.1	Model-to-proof workshop	6

1 Turning strange into known

Reductions are the bridge from story to algorithm. Matching models assignment, 2-SAT models binary choices with clauses, and many cover/path problems reduce to matching or flow after the right graph is built.

1.1 Reduction workflow

A reduction maps the original statement to a known model while preserving answers.

Write these two sentences:

```
Given any valid original solution, I can build a valid model solution.  
Given any valid model solution, I can build a valid original solution.
```

If both directions hold, solving the model solves the original problem.

1.2 Bipartite matching model

Use matching when each chosen relation pairs one item from side A with one item from side B, and each item can be used at most once.

Examples:

- applicants to jobs;
- tasks to machines;
- rows to columns;
- people to allowed roles.

Build an edge for each allowed pair. A matching of size k means k choices with no conflicts.

1.3 2-SAT model

Use 2-SAT when each variable is boolean and every constraint can be written as:

```
(a or b)
```

Convert each clause into implications:

```
not a -> b  
not b -> a
```

The formula is satisfiable iff no variable x and its negation $\text{not } x$ are in the same strongly connected component of the implication graph.

The local exercise keeps constraints small, but the model is the same for large Codeforces and ICPC 2-SAT problems.

1.4 Flow model clues

Consider flow when:

- resources move from sources to sinks;
- edges or vertices have capacities;
- the answer asks for maximum number of simultaneously possible choices;
- a minimum cut interpretation appears.

For vertex capacities, split vertex v into $v_{\text{in}} \rightarrow v_{\text{out}}$ with that capacity.

1.5 Modeling checklist

- What are the objects?
- What is a decision?
- What makes two decisions conflict?
- Is the answer maximum matching, max flow, min cut, shortest path, DSU, or satisfiability?
- Can I prove both directions of the reduction?

1.6 2-SAT mini-example

Clause:

```
(x or not y)
```

Implications:

```
not x -> not y
y      -> x
```

Why? If x is false, the clause can only be satisfied by $\text{not } y$. If y is true, then $\text{not } y$ is false, so the clause can only be satisfied by x .

After building all implications, compute strongly connected components. If x and $\text{not } x$ are in the same component, each implies the other, so no assignment can make one true and the other false.

1.7 Matching reduction example

Suppose three students can solve these problems:

```
A: 1,2  
B: 2  
C: 1,3
```

Build left vertices for students and right vertices for problems. A matching of size 3 is an assignment where every student gets a distinct solvable problem. If maximum matching is 2, no complete assignment exists.

1.8 Flow modeling mini-pattern

For capacity on vertices, split each vertex:

```
v_in -> v_out capacity cap[v]
```

Every original incoming edge goes to `v_in`; every outgoing edge leaves `v_out`. This turns a vertex limit into an edge limit, which ordinary max flow can handle.

This “split the object so the constraint becomes an edge” trick appears often in ICPC modeling problems.

1.9 Reduction proof template

Write both directions:

```
Original -> model:  
Given a valid original solution, construct edges/assignments/flows that satisfy  
the model constraints.  
  
Model -> original:  
Given a valid model solution, translate selected edges/variables/flows back to  
valid original decisions.
```

If either direction fails, the graph you built is only a heuristic.

2 Exercises

2.1 Model-to-proof workshop

For every reduction, prove both directions: a valid original solution maps to a valid graph/logic solution, and a returned graph/logic solution maps back. Draw left vertices, right vertices, and compatibility edges before matching; for 2-SAT, write both implication edges for every clause; for flow, state what one unit means and what each capacity forbids. Test an isolated vertex, contradictory clauses, and a resource with capacity greater than one. Run `CP_TARGET=solution python3 sections/57_reductions_and_modeling/check.py`.

- A. Matching Model: Given a bipartite graph, print maximum matching size.
- B. Small 2-SAT: Given clauses (`a or b`) over signed variables, decide satisfiability.
- C. Capstone Matching: Another matching instance to reinforce the reduction pattern. # Contest checklist

State the invariant, choose the model, prove the operation, then code the smallest reliable version.